Name: Ragyam Sinha
Stream: CSE-B
Semester: 5th
Roll : 13000123064
Sub : Software Eng.
Sub code : ECS 501

Group-A

1.

(i) D) All of the mentioned above
(ii) D) All of the mentioned above
(iii) D) All of the mentioned above
(iv) D) All of the mentioned
(v) B) Low coupling, high cohesion
(vi) A) True
(vii) A) Association
(viii) A) project manager
(ix) A) computer Aided Software Engineering
(x) A) 0

Ragyam Sinha
13000123064
Group-B

2.
→ The Rayleigh curve is a graphical representation used in Software Reliability Engineering (SRE) that shows how software failure rate changes over time during the testing and debugging phases.

It starts high, decreases as bugs are fixed and finally becomes stable as the software matures

→ In the beginning, many defects are found - high failure rate.

→ As testing continues, bugs are fixed - failure rate decreases.

→ After enough testing, very few new defects appear - low and stable failure rate.

Application

a) used in software reliability prediction
b) Helps in release planning
c) use in resource allocation for testing team
d) Applied in models like Rayleigh-norden model for software effort estimation
e) Help decide when the S/W is ready to release

3.
→ The Basic COCOMO (Constructive cost Model) is a Software cost estimation model which estimates efforts, development time and cost based mainly on the size of the software measured in KLOC

→ It is the simplest form of the COCOMO family

→ It uses eqn to estimate efforts and development time.

$$Efforts = a \times (KLOC)^b$$

$$T_{dev} = C \times (Effort)^d$$

→ The model depends on the project type
a) organic - small, simple project
b) Semi-detached - medium project
c) Embedded - complex, real-time system.

Feature

→ Quick and easy way to estimate cost
→ useful in early project planning
→ Helps allocate staff & schedule development time.

Limitations

→ Based only on KLOC, does not consider details factors.
→ less accurate for modern, components - based/ agile project.

6)
→ white Box Testing is a testing method where the internal code, logic and structure of the software are tested.

The tester knows the source code and design test cases based on program flows.

It can be consider as verification, we go inside a system & check how actual functionalities is performed.

→ Focuses on how the S/W works internally
→ Test cases are created using control flow, data flow, loops, conditions and paths in the code.
→ Ensures that every line, branch and path is executed at least once.

Why FPA is better than LOC method

Rajyam Sinha
1300012306S4

Feature	FPA	LOC
Basis	user functions	lines of source code
language	Independent	lang. dependent
when usable	Early design phase	Applied only after coding
Focus	functionality delivery	quantity of code written
Accuracy	More consistent	varies by coding style

by EI: 10 low
EO: 12 high
EQ: 12 avg
ILF: 20 low
ELF: 15 high
VAF: 1.10

Step 1:- use standard Ifpug weight

Function Type	low	Avg	high
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

Step 2:- compute UFP

EI = 10 × 3 = 30
EO = 12 × 7 = 84
EQ = 12 × 4 = 48
ILF = 20 × 7 = 140
EIF = 15 × 10 = 150

Total UFP = 30 + 84 + 48 + 140 + 150 = 452

Step-3: compute Adjusted function point (AFP)

AFP = UFP × VAF
= 452 × 1.10 = 497.2

Adjusted FP ≅ 497

Group - C

Rangam Sinha
13000123064

8) a) Function point Analysis (FPA) is a method to measure the size of software based on the functionality provided to the user, not on the no. of line of code.

→ It counts what the system does from the user's point of view.

→ FPA assumes, more user functionality = more work = large software size.

Step to apply FPA methodology :-

1. Identify system components
   a) External Inputs (EI)
   b) External outputs (EO)
   c) External Inquires (EQ)
   d) Internal logic Files (ILF)
   e) External Interface files (EIL)
2. Classify each component as low, Average or high complexity
3. Assign weights using standard IFPUG tables
   a) EI : 3, 4, 6
   b) EO : 4, 5, 7
   c) EQ : 3, 4, 6
   d) ILF : 7, 10, 15
   e) EIF : 5, 7, 10
4. Calculate unadjusted function point (UFP) for each category.

$$UFP = \sum_{1} (\text{no. of functions} \times \text{weight})$$

5. Compute value adjustment factor (VAF)

$$VAF = 0.65 + 0.01 (\Sigma f_i)$$

6. Compute Adjusted function points (AFP)

$$AFP = UFP \times VAF$$

Rayyan Sinha
1300012306 4

9. a) coupling

→ It is the degree of inter dependence between modules
→ It shows how much one module depends on another
→ High coupling means modules are heavily dependent which is bad design.
→ So low coupling is preferred.

cohesion

→ It is the degree to which elements inside a module belong together.
→ It shows how strongly related the responsibility of a single module are.
→ Low cohesion means module does many unrelated things which makes it bad design.
→ So high cohesion is preferred.

Types of coupling :-

a) Data coupling : Modules communicate by passing only necessary data

b) Stamp coupling : Entire data structure are passed though only part is used

c) control coup : one module control the behaviour of another by passing control info.

d) External coup : Modules depend on external interface, device or protocol.

e) common coup : Modules shared global data

f) content coup : One module directly access or modifies internal data / code of another.

Satyam Sinha
1800012s064

Ans (b) why low coupling and high cohesion good design

(i) Easier to understand
→ High cohesion means each module does one clear task,
→ low coupling means modules are loosely connected

(ii) Easier to maintain & modify
→ if modules are independent, changing one module
→ affect other.

(iii) Better Reusibility
→ High cohesive modules can be reused in other project because they have a single well-defined purpose
→ Low coupling makes it easier to plug in modules into different system.

(C) Function point calculation

given EI = 30
EO = 62
EQ = 24
ILF = 8
EIF = 2

we use standard avg weight

Function Type	Avg weight
EI	4
EO	5
EQ	4
ILF	10
EIF	7

Step-1 : calculate UFP

EI = 30×4 = 120
EO = 62×5 = 310
EQ = 24×4 = 96
ILF = 8×10 = 80
EIF = 2×7 = 14

UFP = 120+310+96+80+10
= 620

Rajyam Sinha
1300123064

Step-2 :- compute VAF

The are 14 general system characteristics

So $\Sigma fi = 14 \times 3 = 42$

$VAF = 0.65 + 0.01 (\Sigma fi)$

$= 0.65 + 0.01 \times 42$

$= 1.07$

Step-3 :- compute Adjusted function point (FP)

$FP = UFP \times VAF$

$= 620 \times 1.07$

$= 663.4$

So Adjusted FP = 663FP

10.

"Regression Testing"

Regression testing is carried out to ensure that previously deployed and tested software still functions as expected after recent changes such as bug fixes, updates or new feature addition.

Explanation:

→ when code is modified, old functionalities may stop working

→ Regression testing re-runs previously passed test cases to confirm that everything still works as expected.

→ It is done multiple times during development especially after each build.

Purpose:-

→ Maintain software stability and reliability

→ Ensure unchanged features still behave correctly.

Rayyan Sinha
1300012306H

Example - If a login module was working before password reset feature was added, regression testing checks that login still works properly.

"Alpha Testing"

It is the first stage of acceptance testing performed by internal testers or selected employees inside the company before product is released to real user

Explanation

→ It is carried out in controlled environment.
→ Testers try to identify bugs, crashes, usability issue and missing features.
→ It is performed by developer or internal domain experts.

purpose :-

→ Detect major defdetect early
→ Ensure that S/w is stable enough for real user testing.

Example :-

An in-house QA team at Google test a email feature inside the company before releasing it to real user.

"Beta Testing"

It is the final stage of testing, performed by real user in the real environment before the full product releases.

Explanation

→ S/w is given to a limited no. of actual users
→ They use the S/w under real-life conditions
→ Their feedback help identify issue that were not found internally.

Rajgopal Sinha
1300123064

Purpose :-

→ validate product performance in real environment
→ collect user feedback.
→ Identify remaining bugs before final release.

Example

A group of selected external user Test a pre-release version of the whatsApp to try new features and report it's bug before the final release.

Question paper year : 2023 Rajyam Sinha
13000123064

Group A

1. a) Black-box
b) program Evaluation and Review Technique.
c) Grady Booch, Ivar Jacobson, James Rumbaugh
d) Spiral Model
e) A Tabular Technique used to present complex logic or rules, showing cond^n and the resulting actions.
f) control flow graph
g) ILF and EIF
h) class name in italics or (Abstract)
i) operational feasibility
j) common coupling.

Group-B

2.
→ A feasibility study is an early analysis done to determine wether a S/w project is practically achievable and worth implementing.

It checks if the project is technically possible, economically beneficial and operationally acceptable before development begins.

Types of feasibility studies :-

1. Technical Feasibility :- check wether the required tech., hardware and software are available and capable of meeting requirements.
2. Economic Feasibility :- Analyze the cost-benefits to see if the system is financially worthwhile.
3. operational Feasibility :- Examines wether the system will work in the real environment and be user-friendly.

Raqiyam Sinha
13000123064

4. Legal Feasibility :- Ensures the system follows all law and regulation

5. Schedule Feasibility :- Determines if the project can be completed with in the given time frame.

6. Organization Feasibility :- checks weather the system fits the culture, structure and goals of the organizata

3.

→ A data dictionary is a central repository that stores detailed information about all data elements used in a system.

It describes data name, type, size, formats, relationship and constraint.

purpose :-

a) Ensure consistency : Developers use same definations for data items.

b) Support Documentation : Acts as official documentation for the system data.

c) Helps in maintainance.

Example :-

Name	Type	Description	Source	Destination	Format
User-Record	Data Store	Store data about every users	process generate ID	process : search display	

Kalyan Sinha
13000123064

5.
→ A function point (FP) is a unit used to measure the size and functionality of a software sys. from the user's point of view.

It count what the system does rather than how many lines of code it has.

a) Internal logical file (ILF)
→ ILF is a logical group of data that is maintained, stored and updated inside the system
→ The system is responsible for creating, editing or deleting this data.

b) External Interface file (EIF)
→ Logical file used by the system but maintained by another external system.
→ The system only reads this data: it does not update it.

Group - C

7. a) Logical DFD :- It shows what the system does processing on the flow of data & process without considering how they are implemented.

Physical DFD :- It shows how the system is actually implemented. including hardware; S/w files. and people involve the process.

Advantage of physical DFD over logical DFD:
1. Shows actual devices, program, files and people involved.
2. Help in system implementation and design
3. Help in estimating time, cost and resources

Rajyam Sinha
7300012306 4

b)

Feature	Logical DFD	Physical DFD
Meaning	Show what the system does	Show how the system works
Focus	Business processes	Technical implementation.
Includes	process, data flow, data store	Hardware, files, program devices
Level	High-level, conceptual	Low-level, detailed
purpose	understand requirement	Design the actual system.

8)

→ coding standards and guidelines are rules and bugs.

(a) practices that developers follow to write clean readable and maintable code.

They provide consistency, quality and reduce error.

common coding standards and guidelines -

a) Naming conventions :-

→ use meaningful names for variable, func and classes

→ follow convention like camel case, snake-case, pascal case.

b) Indentation and Formatting :-

→ proper indentation improves readability

→ consistent spacing, brackets and line breaks make code easy to understand.

c) comments and Documentation :-

→ write comments for complex logic & explain logic

→ use header comments for methods, helps make code easy to understand.

Ragyam Sinha
7300123064

d) code structure and modularity.
→ Break code into short, reusable fun.
→ Avoid long method or repeated code.

e) Error handling guidelines
→ use proper exception handling
→ Avoid silent failures.

f) optimization guidelines
→ Reduce redundant loops and operations
→ use efficient data structures.

b) Software Testing process is the sequence of steps to ensure the developed software meets user requirement and is defect-free.

Major Testing Activities

(i) Testing planning
→ Identify what to test, how to test and resource needed.
→ selecting testing tools.

(ii) Test case design
→ create test cases, test data and test scripts
→ Identify i/p values, expected O/P and test cond^n.

(iii) Test Execution
→ Run the prepared test cases

(iv) Defects logging and Reporting
→ Any failed test becomes a defects
→ Develope fix the bug → Testers.

(V) Test closure
→ prepare a test summary report

9. a)

→ pert (program Evaluation and Review Technique)

→ A project management tool used for planning scheduling and controlling project tasks.

→ Focuses on time estimation by identifying the min time required to complete a project

→ uses a network dig. showing tasks, event and dependence.

⇒ Each task has 3 time estimation

- optimistic time(O) - min time
- pessimistic time(P) - max "
- Most likely Time(M) - probable time

Expected Time = (O+4M+P)/6

→ Helps identify the critical path - the seq of activities that determines projects.

Gantt chart

→ It is a bar chart that represent the Schedule of a project.

→ shows task on the vertical axis and timeline on the horizontal axis

→ Each task is represented by a horizontal bar showing start and end dates

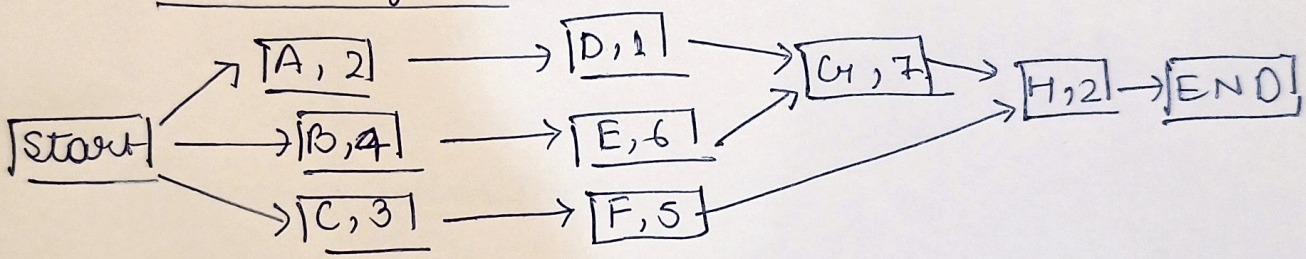
→ Helps track progress, overlaps and sequence of activities

Ragyan Sinha
13000123064

b>

Task	O	M	P	TE $(\frac{O+4M+P}{6})$	variance $(\frac{P-O}{6})^2$
A	1	1	7	$(1+4*+7)/6 = 2$	$((7-1)/6)^2 = 1$
B	2	4	7	$= 4$	$= 0.69$
C	1	2	8	$= 3$	$= 1.36$
D	1	1	1	$= 1$	$= 0$
E	2	5	14	$= 6$	$= 4$
F	2	5	8	$= 5$	$= 1$
G	3	6	15	$= 7$	$= 4$
H	1	2	3	$= 2$	$= 0.11$

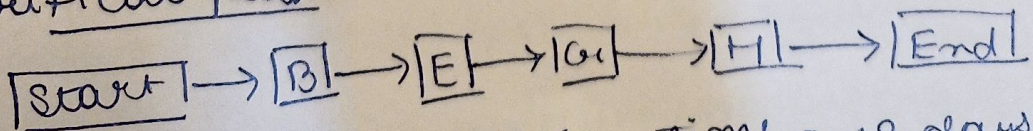
Activity diagram



MT = max(12, 19, 10) = 19

Activity	D	ES	EF = ES + D	LS	LF = LS + D	ST
A	2	0	2	7	9	7
B	4	0	4	0	4	0
C	3	0	3	9	12	9
D	1	2	3	9	10	7
E	6	4	10	4	10	0
F	5	3	8	12	17	9
G	7	10	17	10	17	0
H	2	17	19	17	19	0

Critical path



Expected project completion time = 19 days

Expected project variation = 1 + 4 + 4 + 0.1 = 9.11

≈ 3.02 days

Raghav Sinha
13000123054

Question paper year: 2024

Group-A

1. i) A node (a)
ii) Increases (b)
iii) Acceptance Testing (a)
iv) Five (c)
v) Risk management (b)
vi) Two
vii) A working product (a)
viii) Two
ix) Interface (c)
x) 14 (d)

Group-B

2. **Waterfall Model:**
→ Simplest, linear, sequential SDLC.
→ It is rigid and work when requirement are clear & fixed
→ Used for small to medium size project.

prototyping model:-
→ This model built a quick, working mockup of the system so users can see and refine requirement early
→ It is interactive and flexible allowing changes during development.

Adv of prototyping model over waterfall are-

a) **Better Requirement understanding:-**
→ Users see a working early model, which helps clarify requirement.
→ Waterfall often suffer from unclear or misunderstood area.

Raqyam Sinera
13000123064

b) Early Feedback from user :-

→ users interact with the prototype and give immediat feedback.

c) Reduce risk of failure

→ Mistakes are identified early when the prototype is reviewed.

→ waterfall detects major issues much latter.

d) Helps in Handling changing requirements

→ prototype welcomes changes and refinements during development

→ waterfall is rigid and not suitable for changing requirement

3.

verification

→ validation is the process of checking whether the S/w is built correctly.

→ It focuses on review, inspection and static analysis to ensure requirement, design & code are correct befor execute.

validation

→ validation check wether the final S/w meets the user's actual needs and exception

→ It focuses on testing and dynamic execution of the s/w to ensure it behave correctly

→ In simple word's " Are we building the right product"

Rajyam Sinha
13000123064

4.)

→ A structured chart is graphical design tool used to represent the hierarchical structure of a s/w system.

→ It breaks the system into modules, showing how they are organized and how they interact

Advantages:-

- Improve program structure and modularity
- Helps in top-down design and easier maintainance
- Reduce complexity.

Group-C

7.

→ Integration testing

→ Tests how modules works together after individual unit testing.

→ focuses on interfaces, data flow and communication b/w modules.

System testing

→ Test the entire software system as a whole

→ checks functional, non-func performance, security etc.

Functional testing

→ It verifies that the system behaves a/c to the functional requirement

→ It checks each function by giving input and comparing the actual output with expected output.

Raghavendra
1300012306

How does a test case help in testing
→ Defines exact steps, input and expected result
→ Ensures all func^ns are tested consistently
→ Makes testing repeatable and measurable.
→ provides a records for regression testing later.

Should developer be involve in testing
partially yes, but with limitation.

And

→ developer understand code well and can fix issue later.
→ help in unit testing and debugging.

Error

- A mistake made by developer during coding, design or logic.
- Happens internally before execution.

Bug

- found during testing or execution due to an existing error in the code.
- visible to test or users.

8.
→ Agility means the ability of a s/w team to respond quickly to changes in requirement, Technology or customer needs.

→ It focuses on flexibility, quick delivery, continious improvement and customer involving throughout development.

Agile process :- Development approach that delivers software in small, incremental releases instead of one large final product. It emphasizes short iterations, continious testing, customer feedback and team collaboration.

Features

→ Iterative and incremental development
→ continious feedback from user.
→ working s/w delivery early and freq

4 values and 12 principles

values

1. Individual and interaction over process and tools
2. working s/w over comprehensive documentation
3. customer collaboration over contact negotiation.
4. Responding to change over following a plan

11.

→ Factors that decide the sucess of a project
1. Meeting all user requirements.
2. complete the project within time.
3. completing on project within budget
4. Delivering good quality software.

How s/w project are different from other project

1. Software is intangible
2. Requirement change frequently
3. Had to estimate time and cost accurately.

Raajyam Sinha
13000123064

Characteristics of S/w project

1. Unique for every customer and requirements
2. Highly people-dependent
3. Requirements are volatile
4. S/w needs continuous maintenance after deliver
5. Quality is difficult to measure precisely.

Quantative problem

Ragyam Sinha
13000123064

1.) KLOC = 32 KLOC

$a = 2.4$, $b = 1.05$, $c = 2.5$, $d = 0.38$

Effort (E) $= a \times (KLOC)^b$

$= 2.4 \times (32)^{1.05}$

$= 91.3$ person-months

Development time (T) $= c \times E^d = 2.5 \times (91.3)^{0.38}$

$= 14.1$ months

Avg. staffing $= E/T = \frac{91.3}{14.1} = 6.5$ people $\approx$ 7 people

2.) Line of code = 40 KLOC

EAF = 1.2 ($a = 3.0$, $b = 1.12$, $c = 2.5$, $d = 0.35$)

Effort (E) $= a \times (KLOC)^b \times EAF$

$= 3 \times (40)^{1.12} \times 1.2 = 224.8$ PM

DT (T) $= c \times (E)^d = 2.5 \times (224.8)^{0.35}$

$= 13.9$ months

Productivity $= \frac{40 \text{ KLOC}}{224.8 \text{ PM}} = 0.178$ KLOC/PM

3.) Line of code = 50 KLOC

$a = 2.4$, $b = 1.05$

cost/PM = ₹60000

Effort (E) $= a \times (KLOC)^b = 2.4 \times (50)^{1.05}$

$= 134.8$ PM

cost = Effort $\times$ cost/PM $= 60000 \times 134.8 = ₹80,88,000/-$

4.) COCOMO II: SF = 4.0, EAF = 1.1, A = 2.94

Exponent (E) $= 0.91 + 0.01 \times SF = 0.91 + 0.01 \times 4$

$= 0.95$

Effort $= A \times (size)^E \times EAF$

$= 2.94 \times (size)^{0.95} \times 1.1$

$= 3.24 \times (size)^{0.95}$

Raajyam Singha
1300123064

5.) project size = 60 KLOC

Effort (E) = 150 PM, C = 2.5, d = 0.38

Development Time (Tdev) = C × (effort)^d

= 2.5 × (150)$^{0.38}$

= 17.1 months

6.)

FP calculation

unadjusted function point (UFP)

ELs : 3 (simple) = 3 × 3 = 9

EOs : 4 (avg) = 4 × 5 = 20

EQs : 2 (complex) = 2 × 6 = 12

ILF : 5 (avg) = 5 × 10 = 50

EIF : 3 (complex) = 3 × 10 = 30

Total UFP = 9 + 20 + 12 + 50 + 30 = 121

Adjusted function point (AFP) = 121 × 1.1 = 133.1

7.) given : 200 FP, 50 LOC / FP

∴ Total LOC = FP * LOC/FP

= 200 × 50 = 10,000 LOC = 10 KLOC

8.) given : Total FP = 120, productivity = 6 FP / PM

∴ Effort = total FP / productivity = 120/6 = 20 PM

9.) Modules = 5, 8, 12, 20 and 10 KLOC

TE = 45 PM

Total size = 5 + 8 + 12 + 20 + 10 = 55 KLOC

Avg productivity = Total size / total effort

= 55 KLOC / 45 pm = 1.22 KLOC / PM

10.) Effort = 25 PM, productivity = 500 LOC / PM

∴ Total LOC = Effort × prod

= 25 PM × 500 LOC / PM

= 12500 LOC = 12.5 KLOC

Ranjan Sinha
1300012306H

11) Expected time (TE) = $\frac{O + 4M + P}{6}$

$T1 = \frac{2+4\times4+8}{6} = 26/6 = 4.33$

$T2 = \frac{3+20+9}{6} = 5.33$

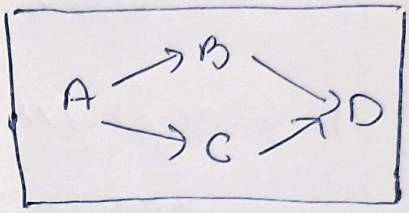
$T3 = \frac{4+24+10}{6} = 6.33$

12) Activity A : $t_e = \frac{2+4\times4+6}{6} = 4$

" B $t_e = \frac{3+4\times6+9}{6} = 6$

" C $t_e = \frac{4+4\times6+8}{6} = 6$

" D $t_e = \frac{2+4\times5+8}{6} = 5$



path from start to finish

- A → B → D = 4+6+5 = 15
- A → C → D = 4+6+5 = 15

Critical path & Duration

- critical paths : A → B → D and A → C → D
- Total expected project = 15 time unit

13) project duration (sequential Tasks)

Duration = sum of all tasks time

= 5+8+10+6+4 = 33

14) project variance :-

variance for a Task = $\left(\frac{P-O}{6}\right)$

O = optimistic
P = pessimistic

A (2,4,6) → O = 2, P = 6

$\sigma^2 = \left(\frac{6-2}{6}\right)^2 = 0.44$

B (3,6,9) $\sigma^2 = 1$

C (4,6,8) $\sigma^2 = 0.44$

D (2,5,8) $\sigma^2 = 1$

Rajyaan Sinha
13000123064

For path A→B→C

Total variance = 0.44 + 1 + 1 = 2.44

For path A→C→D

Total variance = 0.44 + 0.44 + 1 = 1.88.

15) project slack time

Slack = Total project time − critical path time

= 30.25 weeks.

16) System Availability

MTTF = 200 hrs

MTTR = 20 hrs

$A = \frac{MTTF}{(MTTR + MTTF)} = \frac{200}{220} = 0.909 = 90.9\%$.

17) Reliability ~~CRF~~ R(t)

$R(t) = (e)^{-\lambda t} = e^{(-0.002 \times 200)}$

$= e^{-0.4} = 0.67 = 67\%$.

probability of failure free operation for 200 hrs

18) Failure intensity = $\frac{\Delta \text{failure}}{\Delta \text{time}} = \frac{5-3}{200-100} = \frac{2}{100} = 0.02$ flrs

19) Risk Exposure (RE) = probability * loss = 0.3×400000

= ₹ 120000 00/-

20) Risk mitigation

new RE = 0.15×400000

= ₹ 60,000

% reduction = $\frac{120000 - 60000}{120000} \times 100$

= 50%.

21) Defect density = total defects / size

= 50 / 10000 = 0.005 defects / LOC

= 5 defect / KLOC

Rajyam Sinha
1300012306H

22) productivity (LOC / PM)

Total effort = 3 developers × 4 months
= 12 PM

productivity = $\frac{6000}{12}$ = 500 LOC / PM

23) Effective new LOC

Reusable code = 10% of 20000 LOC

= $\frac{50}{100}$ × 20000 = 6000 LOC

new LOC to be developed = 20000 − 6000
= 14000 LOC

24) productivity (FP / PM)

productivity = Delivered FP / Effort

= 200 / 40 = 5 FP / PM

25) Maintainability Index (MI)

MI = 171 − 5.2 ln(HV) − 0.23 × CC − 16.2 × ln(LOC)

= 171 − (5.2 × 7.31) − (0.23 × 10) − (16.2 × 7.6)

= 171 − 38 − 0.2 − 123 ≈ −3.4

The negative value suggest very low maintainability